Ch 8.1: Decision Trees

Lecture 24 - CMSE 381

Prof. Guanqun Cao

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Wed, Oct 29, 2025

Announcements

Last time:

- Cubic Splines

This lecture:

- 8.1 Decision Trees

Announcements:

- HW #7 due Sun, 11/2
- Projects

	F	10/17	***Review***		
	M	10/20	Fall Break		
	W	10/22	**Midterm #2**		
21	F	10/24	Polynomial & Step Functions	7.1-7.2	HW #5 Due Sun 10/28
22	M	10/27	Step Functions; Basis functions; Start Splines	7.2-7.4	
23	W	10/29	Regression Splines	7.4	
24	F	10/31	Decision Trees	8.1	HW #6 Due Sun 11/2
25	M	11/3	Random Forests	8.2.1, 8.2.2	
26	W	11/5	Maximal Margin Classifier	9.1	
27	F	11/7	SVC	9.2	HW #7 Due Sun 11/9
28	M	11/10	SVM	9.3, 9.4	
29	W	11/12	Single Layer NN	10.1	
30	F	11/13	Multi Layer NN	10.2	HW #8 Due Sun 11/16
31	M	11/17	CNN	10.3	
32	W	11/19	Unsupervised learning / clustering	12.1, 12.4	
33	F	11/21	Virtual: Project Office Hours		HW #9 Due Sun 11/23
	M	11/24	***Review***		
	W	11/26	**Midterm #3**		
	F	11/28	Thanksgiving		
	M	12/1	Virtual: Project Office Hours		
	W	12/3	Virtual: Project Office Hours		
	F	12/5			**Project Due**

Section 1

Decision Trees

Big idea

- Previously, we've always had some form of function for prediction: $f(X_1, \cdots, X_p) = \hat{Y}$
- Now, we make a sequence of decisions to make a prediction, either regression or classification version
- remind them regression means the outcome is quantitative, classification qualitative
- (at some point could mention this is more similar to step function fitting)

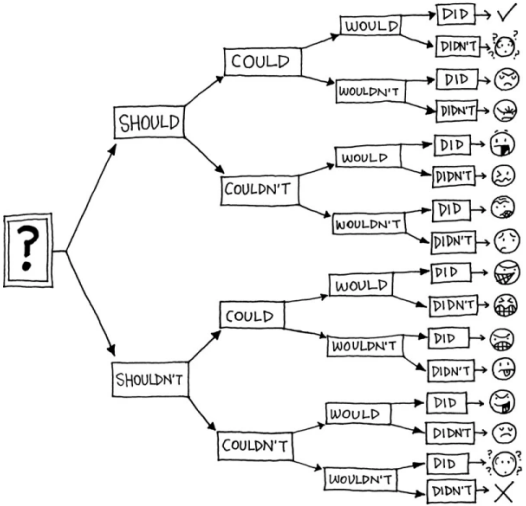


Image: https://marekbennett.com/2014/02/14/decision-tree/

Subset of Hitters data

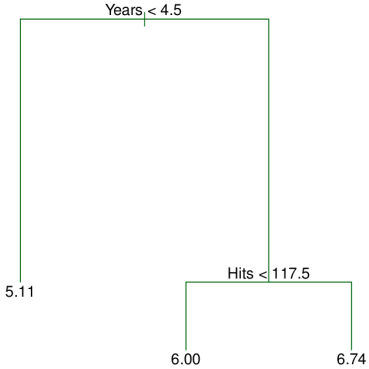
	Hits	Years	Salary	LogSalary
1	81	14	475.0	6.163315
2	130	3	480.0	6.173786
3	141	11	500.0	6.214608
4	87	2	91.5	4.516339
5	169	11	750.0	6.620073
...	...	...	...	...
317	127	5	700.0	6.551080
318	136	12	875.0	6.774224
319	126	6	385.0	5.953243
320	144	8	960.0	6.866933
321	170	11	1000.0	6.907755

- Remove observations missing salary values
- log transform salary for something closer to bell shape
- Goal: predict log salary (can reverse by returning $\exp(x)$ if model returns x)

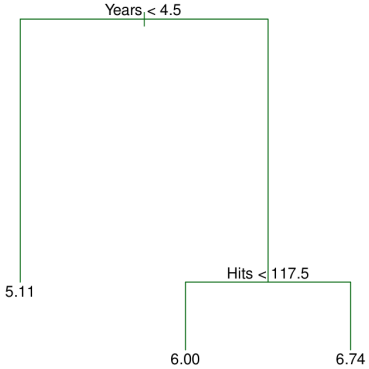
First decision tree example

	Hits	Years	LogSalary
1	81	14	6.163315
2	130	3	6.173786
3	141	11	6.214608
4	87	2	4.516339
5	169	11	6.620073
...	...	...	...
317	127	5	6.551080
318	136	12	6.774224
319	126	6	5.953243
320	144	8	6.866933
321	170	11	6.907755

- Top split assigns observations with *Years* < 4.5 (in major leagues) to left branch
- Return mean response for players with that.
- Predictions:
  - mean log salary is 5.107, so returns $\exp(5.107) = \$165.174$ thousand dollars
  - $5.999 \Rightarrow \$402,834$
  - $6.740 \Rightarrow \$845,346$



Interpretation of example



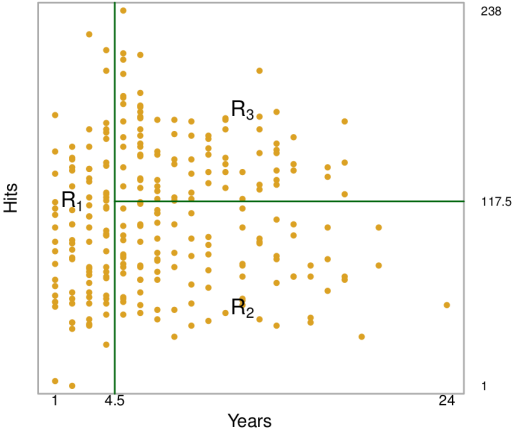
- Years most important factor for determining salary
- Players with less experience earn lower salaries than more experienced
- For the less experienced players, number of hits plays little role in salary
- For more experienced players, number of hits affects it
- Likely an oversimplification of real relationship, but easier to interpret and has nice graphical representation

Coding a regression decision tree

Regions defined by the tree

- Point out tree is "upside down"
- Leaves (terminal nodes) of the tree:
  - $R_1 = \{X \mid Years < 4.5\}$
  - $R_2 = \{X \mid Years >= 4.5, Hits < 117.5\}$
  - $R_3 = \{X \mid Years >= 4.5, Hits >= 117.5\}$
- Other splits are called Internal Nodes
- Segments that connect nodes called Branches or Edges

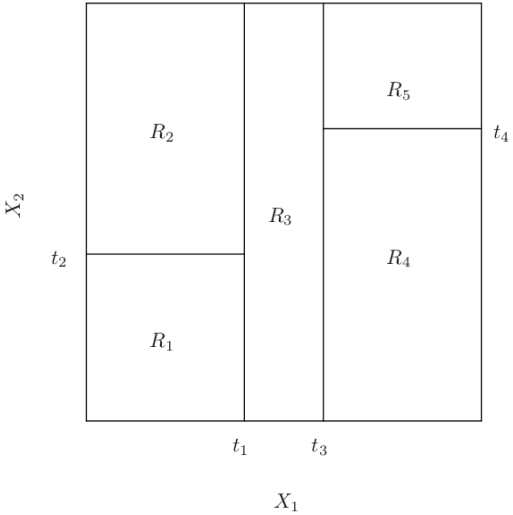
Viewing Regions Defined by Tree



How do we actually get the tree? Two steps

1. We divide the predictor space – that is, the set of possible values for $X_1, X_2, \cdots, X_p$ — into J distinct and non-overlapping regions, $R_1, R_2, \cdots, R_J$.
2. For every observation that falls into the region R_j, we make the same prediction = the mean of the response values for the training observations in R_j.

Here's probably a good place to mention the parallel with step function.



Step 1: How do we decide on R_js?

Goal:

Find boxes $R_1, \cdots, R_J$ that minimize

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

$\hat{y}_{R_j}$ = mean response for training observations in jth box

- cost is just RSS
- Can't actually check every possible partition
- Instead, go for top-down greedy approach called *Recursive binary splitting*
- Begins at top of tree with all data points
- Uses best split at every step

Recursive Binary Splitting

One split:

- Pick X_j and cutpoint s
- so that splitting into $\{X \mid X_j < s\}$ and $\{X \mid X_j \geq s\}$ results in largest possible reduction in RSS

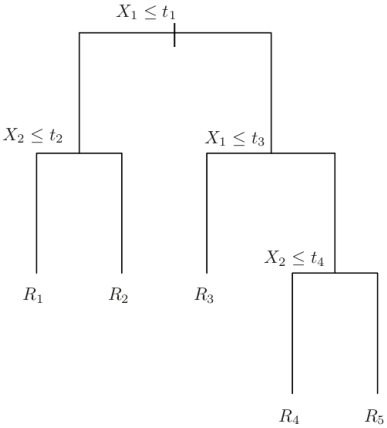
$$R_1(j,s) = \{X \mid X_j < s\}$$
$$R_2(j,s) = \{X \mid X_j \geq s\}$$

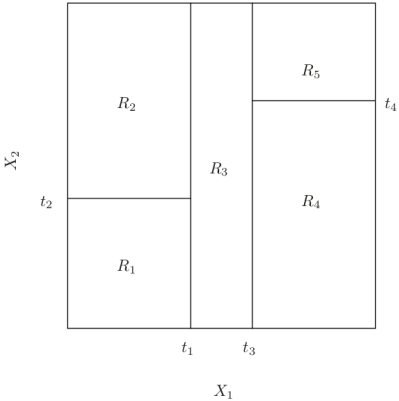
Note here we look at all combos of Xj and s for each split of the tree.

$$\sum_{i|x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i|x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

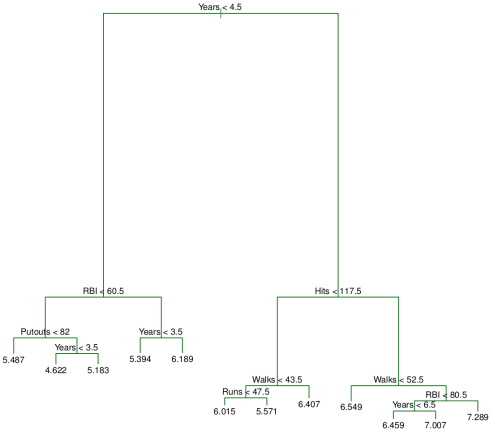
Rinse and repeat

- Do this repeatedly
- Each time can split one of the previously identified regions
- Keep going until some stopping criterion is reached. E.g. until each region has at most 5 observations



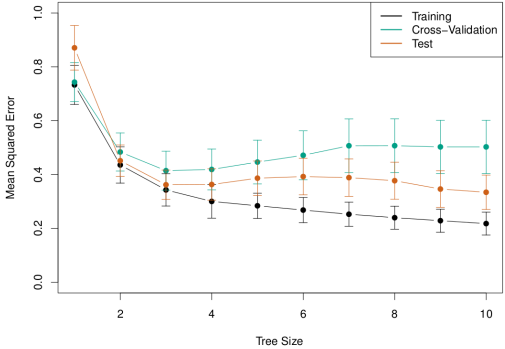


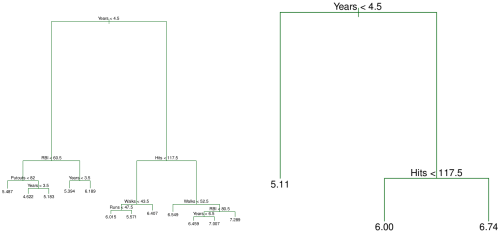
Pruning



- Big trees leave you open to potential overfitting
- Could just stop building earlier, but that's short sighted
- Instead, grow a big tree and prune it back
- Find subtree with best test error rate
- Too many subtrees to test them all

Result of pruning



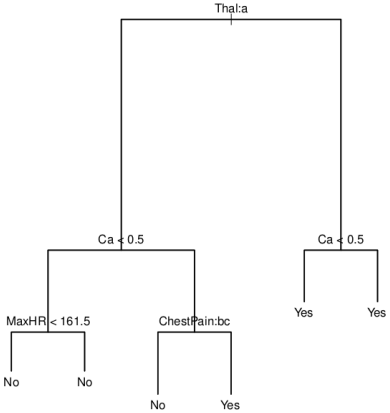


Result of pruning is the three leaf tree on the right. This example technically uses a more complex pruning method than the one described in the slides, but the general idea is the same.

Section 2

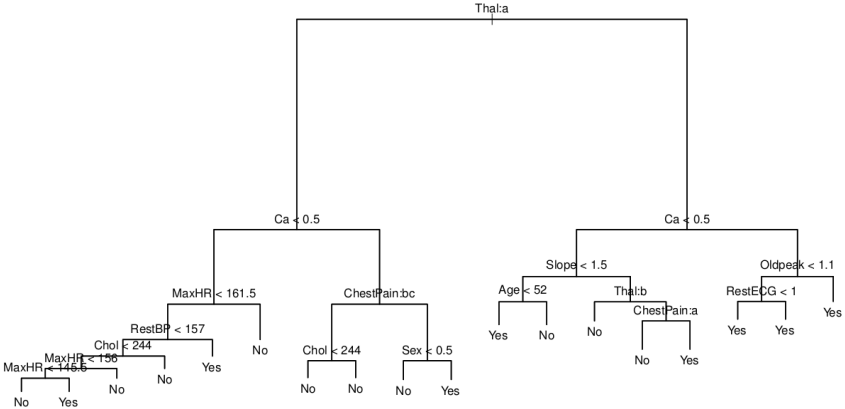
Classification Decision Tree

Basic idea



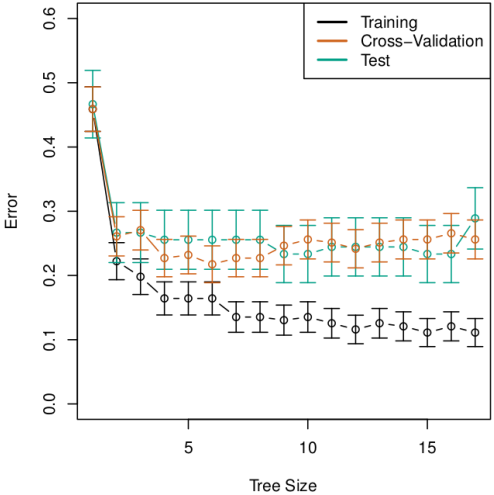
- $\hat{p}_{mk}$ = proportion of training observations in R_m from the kth class
- $E = 1 - \max_k(\hat{p}_{mk})$ *Fraction of training observations not in the most common class*
- Example from heart data
- Can't use RSS, need error rate
- Could decide splits by classification error rate
- Not necessarily the best, you can use other options but I'm going to skip that.

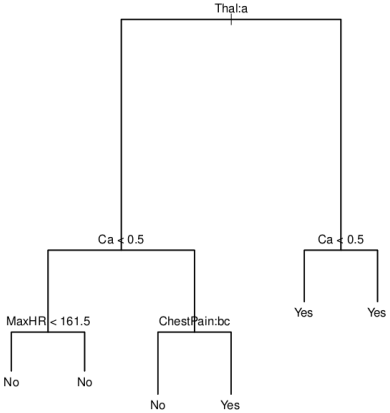
Example



The terminal nodes here: Yes = person has heart disease, No = no heart disease

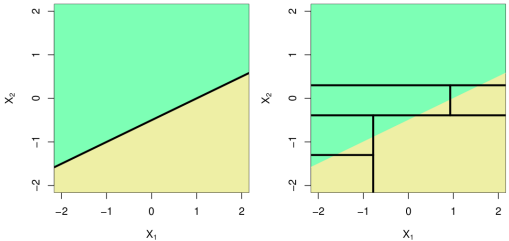
Pruning the example



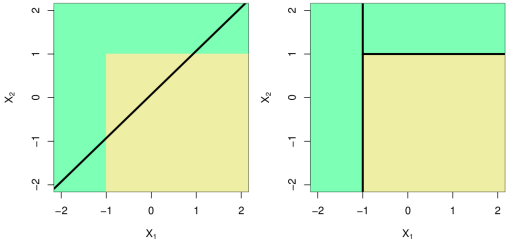


More coding!

Linear models vs trees



Obviously linear does better here



Not going to beat this case though

Pros/Cons

Pros:

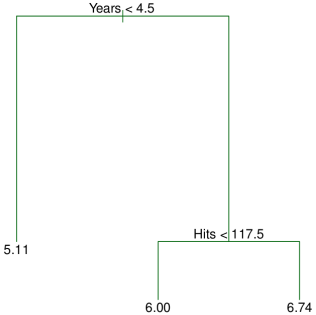
- Trees are very easy to explain to people. Often easier to explain than linear regression!
- Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small).
- Trees can easily handle qualitative predictors without the need to create dummy variables.

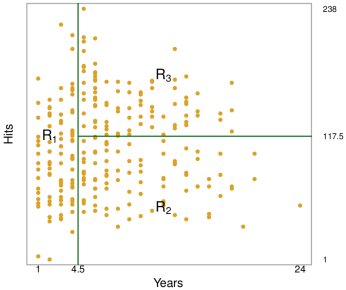
Cons:

- Not as accurate as other methods of classification and regression
- Not robust: small change in data can cause large change in estimated tree
- Fix..... aggregate many decision trees

TL;DR

- Split into regions by greedily decreasing RSS
- Prune tree by using cost complexity
- Not robust - Next time, figure out how to aggregate trees





Next time

	F	10/17	***Review***		
	M	10/20	Fall Break		
	W	10/22	**Midterm #2**		
21	F	10/24	Polynomial & Step Functions	7.1-7.2	HW #5 Due Sun 10/28
22	M	10/27	Step Functions; Basis functions; Start Splines	7.2-7.4	
23	W	10/29	Regression Splines	7.4	
24	F	10/31	Decision Trees	8.1	HW #6 Due Sun 11/2
25	M	11/3	Random Forests	8.2.1, 8.2.2	
26	W	11/5	Maximal Margin Classifier	9.1	
27	F	11/7	SVC	9.2	HW #7 Due Sun 11/9
28	M	11/10	SVM	9.3, 9.4	
29	W	11/12	Single Layer NN	10.1	
30	F	11/13	Multi Layer NN	10.2	HW #8 Due Sun 11/16
31	M	11/17	CNN	10.3	
32	W	11/19	Unsupervised learning / clustering	12.1, 12.4	
33	F	11/21	Virtual: Project Office Hours		HW #9 Due Sun 11/23
	M	11/24	***Review***		
	W	11/26	**Midterm #3**		
	F	11/28	Thanksgiving		
	M	12/1	Virtual: Project Office Hours		
	W	12/3	Virtual: Project Office Hours		